PETWELL

PetWell — คำถามท้ายโปรเจกต์ รอบที่ 2

เก็บตก 9 ข้อที่ยังไม่ผ่าน

อธิบายแบบเข้าใจง่าย เริ่มจากคำตอบสั้นๆ แล้วค่อยลงรายละเอียด ทุกข้ออ้างอิงโค้ดจริงในโปรเจกต์

Next.js 16 · NestJS 11 · PostgreSQL · Prisma 7 · TypeScript

สิงหาคม 2026

Pet Health Passport & Appointment Management System

Contents

1 ข้อ 5 — Token เก็บไว้ที่ไหน ส่งไปยังไง **5**

ตอบสั้นๆ ก่อน . . . 5
บัตรผ่านเกิดขึ้นตอนไหน . . . 5
เก็บไว้ที่ไหน . . . 5
ส่งไป backend ยังไง . . . 5
ทำไมถึงเลือกวิธีนี้ . . . 5
เรื่อง XSS ที่มักถูกถามต่อ . . . 6
ข้อเสียที่ยอมรับไว้ . . . 6

2 ข้อ 6 — ข้อมูลไหนดึงจาก Server ข้อมูลไหนดึงจาก Client **7**

ตอบสั้นๆ ก่อน . . . 7
Server Component คือ . . . 7
ตัวอย่าง Server — หน้าจองนัด . . . 7
ตัวอย่าง Client — ตารางเวลาว่าง . . . 7
จุดที่ควรเน้นตอนตอบ . . . 8
สัดส่วนในโปรเจกต์ . . . 8

3 ข้อ 9 — providers, controllers, imports, exports คืออะไร **9**

เทียบให้เห็นภาพก่อน . . . 9
controllers . . . 9
providers . . . 9
imports . . . 10
exports . . . 10
ตารางสรุป . . . 10
กรณีพิเศษในโปรเจกต์ . . . 10

4 ข้อ 10 — DTO ป้องกันอะไร **12**

ตอบสั้นๆ ก่อน . . . 12
ทำไม TypeScript ช่วยไม่ได้ . . . 12
ตัวอย่างในโปรเจกต์ . . . 12
DTO ทำ 3 อย่าง . . . 12
ถ้าไม่มี DTO จะเกิดอะไร . . . 13

5 ข้อ 11 — Dependency Injection คืออะไร ดียังไง **14**

ตอบสั้นๆ ก่อน . . . 14
หน้าตาในโค้ดจริง . . . 14
ถ้าสร้างเองจะเกิดอะไร . . . 14
DI Container คือใคร . . . 15
เทียบให้เห็นชัด . . . 15

6 ข้อ 12 — ทำไม Controller ไม่ควรเขียน Business Logic 16

ตอบสั้นๆ ก่อน . . . 16
Controller ในโปรเจกต์นี้เป็นบรรทัดเดียวทุกตัว . . . 16
หน้าทีจริงมีแค่ 3 อย่าง . . . 16
ถ้าใส่งานจริงลงไปจะเกิดอะไร . . . 16
การแบ่งชั้นในโปรเจกต์ . . . 17

7 ข้อ 13 — Request เข้ามาแล้วผ่านอะไรบ้าง 18

เทียบเป็นการเข้าตึกสำนักงาน . . . 18
ลำดับเต็ม . . . 18
ตัวอย่างจริง ตอนกดจองนัด . . . 18
ทำไมยามต้องมาก่อนตรวจเอกสาร . . . 18
จุดทีมักเข้าใจผิด . . . 19
เรืองข้อความ error . . . 19

8 ข้อ 14 — unknown ปลอดภัยกว่า any ยังไง 20

ตอบสั้นๆ ก่อน . . . 20
เทียบเป็นกล่องพัสดุ . . . 20
เห็นความต่างในสองบรรทัด . . . 20
ตัวอย่างจริงในโปรเจกต์ . . . 20
ทำไมต้องใช้ unknown ตรงนี้ . . . 20
ถ้าเปลียนเป็น any จะเกิดอะไร . . . 21
ทีอืนในโปรเจกต์ทีใช้ unknown . . . 21
หลักทีใช้ . . . 21

9 ข้อ 15 — Union Type กับ Enum ต่างกันยังไง 22

ตอบสั้นๆ ก่อน . . . 22
เห็นความต่างชัดทีสุดในโปรเจกต์นี้ . . . 22
ตารางเปรียบเทียบ . . . 22
ทำไม Backend เลือก Enum . . . 22

ทำไม Frontend เลือก Union Type 23
ข้อควรระวังของ enum 23
วิธีเลือก 23

1 ข้อ 5 — Token เก็บไว้ที่ไหน ส่งไปยังไง

ตอบสั้นๆ ก่อน

Token คือ **บัตรผ่าน** ที่ระบบออกให้หลังล็อกอินสำเร็จ

PetWell เก็บบัตรใบนี้ไว้ใน cookie ชนิดพิเศษที่ **JavaScript ในเบราว์เซอร์แตะไม่ได้** เรียกว่า httpOnly

เวลาจะเรียก API ฝั่ง server เป็นคนหยิบบัตรมาแนบให้ ผู้ใช้ไม่เคยเห็นบัตรใบนี้เลย

บัตรผ่านเกิดขึ้นตอนไหน

ตอนล็อกอินสำเร็จ backend เป็นคนออกให้

```
const access_token = await this.accessTokenService.sign({
  sub: user.id,
  email: user.email,
  role: user.role,
  clinicId: user.clinicId,
});
```

ข้างในบัตรมีแค่ 4 อย่าง คือ ใครคนนี้ อีเมลอะไร บทบาทอะไร สังกัดคลินิกไหน

ไม่มีรหัสผ่าน และบัตรมีลายเซ็นดิจิทัลกำกับ ใครแก้ตัวเลขข้างในบัตรเอง ลายเซ็นจะไม่ตรงและถูกปฏิเสธทันที

บัตรมีอายุ 24 ชั่วโมง

เก็บไว้ที่ไหน

ฝั่ง Next.js เอาบัตรไปใส่ใน cookie แล้วติดป้าย `httpOnly` ไว้

ป้ายนี้บอกเบราว์เซอร์ว่า **ห้าม JavaScript แตะ cookie อันนี้**

ถ้าลองพิมพ์ `document.cookie` ใน console จะไม่เห็นเลย

ส่งไป backend ยังไง

ทุกครั้งที่จะเรียก API จะผ่านฟังก์ชันนี้ ซึ่งรันบน server เสมอ

```
export async function authFetch<T>(path: string, options = {}) {
  const session = await auth();
  const token = session?.user?.access_token;
  if (!token) redirect('/login');
  return apiFetch<T>(path, { ...options, token });
}
```

แล้วแนบบัตรเป็นหัวจดหมาย

```
newHeaders.set('Authorization', `Bearer ${token}`);
```

ฝั่ง NestJS มียามชื่อ `AuthGuard` ยืนอยู่หน้าทุกประตู คอยดูบัตรก่อนปล่อยเข้า

ทำไมถึงเลือกวิธีนี้

กันการขโมยบัตร — ถ้าเก็บบัตรใน `localStorage` แบบทีหลายคนทำ โค้ดอะไรก็อ่านได้ รวมถึงโค้ดของคนไม่ดีทีแอบเข้ามา แต่ httpOnly ปิดทางนั้นไปเลย

บัตรไม่เคยไปถึงเบราว์เซอร์ — ทุกการเรียก API เกิดบน server เบราว์เซอร์เห็นแค่หน้าเว็บทีทำเสร็จแล้ว

backend ไม่ต้องจำอะไร — ไม่ต้องเก็บรายชื่อคนทีล็อกอินอยู่ แค่ดูลายเซ็นบนบัตรก็พอ
ทำให้เพิมเครืองเซิร์ฟเวอร์ได้ทันทีโดยไม่ต้องแชร์ข้อมูลกัน

เรือง XSS ทีมักถูกถามต่อ

XSS คือการทีคนอืนแอบเอาโค้ดของเขามารันในเว็บเรา

เกิดตอนทีเว็บเอาข้อความทีผู้ใช้พิมพ์ไปทำตามคำสัง แทนทีจะแค่แสดงเฉยๆ ผู้โจมตีจึงพิมพ์คำสังลงไปแทนข้อความธรรมดา

พอโค้ดของเขารันในหน้าเว็บเรา มันทำอะไรก็ได้เท่าทีโค้ดเราทำได้ รวมถึงขโมยบัตรผ่านส่งไปให้เขา

`httpOnly` ทำให้ขโมยไม่ได้ เพราะโค้ดอ่าน cookie ไม่ออก

แต่ต้องพูดให้ตรง — `httpOnly` ไม่ได้กัน XSS มันกันแค่ไม่ให้ขโมยบัตรออกไปได้ ถ้ามี XSS จริง ผู้โจมตียังสังงานในนามเราได้ แต่ทำได้เฉพาะตอนทีเรายังเปิดหน้าเว็บค้างไว้ พอปิดแท็บก็จบ

PetWell ไม่เสียงเรืองนี เพราะ React แสดงข้อความทีผู้ใช้พิมพ์เป็นตัวอักษรเสมอ ไม่เคยเอาไปรันเป็นคำสัง ต่อให้กรอก `<script>` ลงช่อง notes หน้าจอก็โชว์คำว่า `<script>` เฉยๆ

ข้อเสียทียอมรับไว้

บัตรทีออกไปแล้ว ยกเลิกไม่ได้ — ถ้าปิดบัญชีใครกลางคัน บัตรใบเก่ายังใช้ได้จนหมดอายุ เพราะยามดูแค่ลายเซ็น ไม่ได้เดินไปถามฐานข้อมูลทุกครัง

24 ชัวโมงถือว่ายาว สำหรับบัตรทียกเลิกไม่ได้

ถ้าจะพัฒนาต่อ ควรลดอายุบัตรเหลือ 15 นาที แล้วเพิมบัตรใบทีสองไว้ขอบัตรใหม่ ซึงเก็บในฐานข้อมูลและยกเลิกได้

> **สรุปข้อ 5**
> Token คือบัตรผ่านที backend ออกให้ตอนล็อกอิน เก็บใน cookie ชนิด httpOnly ที JavaScript อ่านไม่ได้
> ทำให้ขโมยไม่ได้ และ server เป็นคนแนบให้ทุกครังทีเรียก API
> แลกมากับข้อเสียคือบัตรทีออกไปแล้วยกเลิกกลางคันไม่ได้

2 ข้อ 6 — ข้อมูลไหนดึงจาก Server ข้อมูลไหนดึงจาก Client

ตอบสั้นๆ ก่อน

ถามตัวเองแค่คำถามเดียว

ตอนเปิดหน้านี้ เรารู้แล้วหรือยังว่าต้องใช้ข้อมูลอะไร

รู้แล้ว ให้ดึงบน **Server**

ยังไม่รู้ ต้องรอผู้ใช้กดเลือกก่อน ให้ดึงบน **Client**

Server Component คือ

โค้ดที่รันบนเซิร์ฟเวอร์เท่านั้น ไม่ได้ถูกส่งไปให้เบราว์เซอร์เลย ดึงข้อมูลเสร็จแล้วค่อยส่ง HTML ที่พร้อมแล้วไป

ใน Next.js ทุกไฟล์เป็นแบบนี้โดยอัตโนมัติ ยกเว้นไฟล์ที่เขียน `'use client'` ไว้บนสุด

ตัวอย่าง Server — หน้าจองนัด

```
export default async function BookAppointmentPage() {
  const [pets, clinics] = await Promise.all([
    PetApi.listMine().catch((): Pet[] => []),
    ClinicApi.list().catch((): Clinic[] => []),
  ]);
  return <BookAppointmentForm pets={pets} clinics={clinics} />;
}
```

เหตุผลที่เลือก Server

รายชื่อสัตว์กับรายชื่อคลินิก ยังไงก็ต้องใช้ ไม่มีเงื่อนไข รู้ตั้งแต่ก่อนเปิดหน้า

ต้องใช้บัตรผ่านที่อยู่ใน cookie httpOnly ซึ่งฝั่ง Client อ่านไม่ได้

ผู้ใช้ไม่ต้องเห็นวงกลมหมุนรอโหลด เพราะข้อมูลมาพร้อมหน้าเว็บแล้ว

ตัวอย่าง Client — ตารางเวลาว่าง

```
'use client';

useEffect(() => {
  if (!clinicId || !date) return;
  startLoading(async () => {
    const result = await getSlotAvailabilityAction(clinicId, date);
    setLoaded({ key: requestKey, slots: result?.slots ?? null });
  });
}, [clinicId, date, requestKey]);
```

เหตุผลที่ต้องเป็น Client

ตอนเปิดหน้ายังไม่รู้เลยว่าผู้ใช้จะเลือกคลินิกไหน วันไหน จะดึงล่วงหน้าก็ไม่รู้จะดึงของอะไร

ที่ว่างเปลี่ยนตลอดเวลา ต้องดึงสดตอนที่ผู้ใช้เลือกจริงๆ

ต้องจำสถานะและตอบสนองการคลิก ซึ่งทำได้เฉพาะฝั่ง Client

จุดที่ควรเน้นตอนตอบ

ตารางเวลาว่างเป็น Client Component ก็จริง แต่ **มันไม่ได้ยิงไปหา NestJS โดยตรง**

```
const result = await getSlotAvailabilityAction(clinicId, date);
```

`getSlotAvailabilityAction` เป็น **Server Action** — ฟังก์ชันที่เขียนไว้ให้ Client เรียกได้ แต่ตัวมันรันบน server

เบราว์เซอร์แค่บอกว่า "ช่วยหาที่ว่างให้หน่อย" แล้ว server เป็นคนไปเอาบัตรผ่านมาใช้ให้

ด้วยวิธีนี้บัตรผ่านไม่เคยหลุดออกไปอยู่ในเบราว์เซอร์เลย แม้แต่ตอนที่ Client เป็นคนเริ่มขอ

สัดส่วนในโปรเจกต์

ประเภท	ตัวอย่าง	ทำไม
Server	ทุกไฟล์ `page.tsx`	รู้ล่วงหน้าว่าต้องใช้อะไร ต้องใช้บัตรผ่าน
Client	`BookAppointmentForm`	ต้องจำสิ่งที่ผู้ใช้เลือก
Client	`SlotPicker`	ดึงตามสิ่งที่ผู้ใช้เพิ่งเลือก
Client	`StatusActions`	ปุ่มที่กดแล้วต้องรอผลและแจ้งเตือน

มีไฟล์ที่เขียน `'use client'` อยู่ 24 ไฟล์ จากทั้งหมดราว 115 ไฟล์ ที่เหลือเป็น Server ทั้งหมด

> **สรุปข้อ 6**
> รู้ล่วงหน้าว่าต้องใช้ข้อมูลอะไร ให้ดึงบน Server เพราะเร็วกว่าและใช้บัตรผ่านได้ ส่วนข้อมูลที่ขึ้นกับสิ่งที่ผู้ใช้เพิ่งกดเลือก ต้องดึงบน Client และให้ Client เรียกผ่าน Server Action เสมอ เพื่อไม่ให้บัตรผ่านหลุดออกไปอยู่ในเบราว์เซอร์

3 ข้อ 9 — providers, controllers, imports, exports คืออะไร

เทียบให้เห็นภาพก่อน

มองว่า **โมดูล คือแผนกหนึ่งในบริษัท**

ช่อง	เทียบเป็น
`controllers`	พนักงานต้อนรับ คอยรับเรื่องจากคนนอก
`providers`	พนักงานที่ทำงานอยู่ในแผนกนี้
`imports`	ขอยืมคนจากแผนกอื่นมาช่วย
`exports`	ยอมให้แผนกอื่นมายืมคนของเรา

controllers

ประกาศคลาสที่ทำหน้าที่รับ request เข้ามาในแผนกนี้

```
@Module({
  controllers: [AppointmentController],
  providers: [AppointmentService],
  exports: [AppointmentService],
})
export class AppointmentModule {}
```

`AppointmentController` ดูแล 7 เส้นทาง เช่น `POST /appointments` และ `GET /clinics/:clinicId/availability`

ถ้าลืมใส่ในช่องนี้ โค้ดยังคอมไพล์ผ่าน ไม่มี error ใดๆ แต่ทุก URL ในนั้นจะตอบ 404 เพราะ Nest ไม่เคยรู้ว่ามีไฟล์นี้อยู่

นี่คือความผิดพลาดที่หายากที่สุด เพราะไม่มีอะไรเตือนเลย

Controller **ไม่ต้อง** ใส่ในช่อง `providers` ซ้ำ Nest จัดการให้เองอยู่แล้ว

providers

ประกาศคลาสที่ Nest ต้องสร้างขึ้นมาให้ และแจกให้ใครก็ตามในแผนกนี้ที่ขอ

`AppointmentController` ขอใช้ `AppointmentService` แบบนี้

```
constructor(private readonly appointmentService: AppointmentService) {}
```

ขอได้เพราะ `AppointmentService` อยู่ในช่อง `providers` ของแผนกเดียวกัน

ถ้าลืมใส่ แอปจะไม่ยอม start เลย พร้อมข้อความ `Nest can't resolve dependencies of AppointmentController`

ข้อดีคือรู้ทันทีตั้งแต่ตอนเปิดแอป ไม่ใช่ตอนมีผู้ใช้จริงมาเรียก

ช่องนี้ใช้ลงทะเบียนของพิเศษได้ด้วย เช่นการทำให้ยาม `AuthGuard` ไปยืนหน้าทุกประตูทั้งระบบ

```
providers: [
  { provide: APP_GUARD, useClass: AuthGuard },
]
```

อ่านว่า "ถ้ามีใครขอของชื่อ `APP_GUARD` ให้ส่ง `AuthGuard` ไป" — ชื่อ `APP_GUARD` เป็นชื่อพิเศษที่ Nest รู้จัก พอเห็นแล้วจะเอาไปติดตั้งเป็นยามระดับทั้งแอปให้เอง

imports

ประกาศว่าแผนกนี้ขอใช้ของจากแผนกอื่น

```
@Module({
  imports: [UserModule, JwtModule, HashModule],
  controllers: [AuthController],
  providers: [AuthService, AccessTokenService],
  exports: [AccessTokenService],
})
export class AuthModule {}
```

`AuthService` ต้องใช้ `UserService` เพื่อค้นหาผู้ใช้จากอีเมล จึงต้องยืมมาจาก `UserModule`

exports

ประกาศว่าของชิ้นไหนของแผนกเรา ให้แผนกอื่นยืมได้

`UserModule` เขียน `exports: [UserService]` ไว้ `AuthModule` ถึงยืมได้

ต้องตรงกันเป็นคู่ ถ้าฝั่งหนึ่ง import แต่อีกฝั่งไม่ได้ export ก็ยังใช้ไม่ได้อยู่ดี เหมือนไปขอยืมของแต่เจ้าของไม่ยอมให้

ตารางสรุป

ช่อง	ทำอะไร	ลืมใส่แล้วเกิดอะไร
`controllers`	สร้าง URL	ตอบ 404 แบบเงียบๆ ไม่มีอะไรเตือน
`providers`	ของที่ใช้ได้ในแผนกนี้	แอปไม่ start ฟ้องทันที
`imports`	ยืมของจากแผนกอื่น	ขอใช้ไม่ได้ ฟ้องตอน start
`exports`	ให้แผนกอื่นยืมของเรา	คนอื่นยืมไม่ได้แม้จะ import แล้ว

กรณีพิเศษในโปรเจกต์

`DatabaseModule` ติดป้าย `@Global()` ไว้

```
@Global()
@Module({
  providers: [PrismaService],
  exports: [PrismaService],
})
export class DatabaseModule {}
```

แปลว่า **ทุกแผนกใช้ได้เลยโดยไม่ต้องขอยืม** เหมาะกับของที่ทุกแผนกต้องใช้จริงๆ อย่างการเชื่อมต่อฐานข้อมูล

แต่ไม่ควรใช้บ่อย เพราะพอทุกอย่างใช้ได้หมด จะดูไม่ออกว่าแผนกไหนพึ่งพาอะไรบ้าง

สรุปข้อ 9
มองโมดูลเป็นแผนก — `controllers` คือพนักงานต้อนรับที่รับเรื่องจากข้างนอก `providers` คือคนที่ทำงานในแผนกนี้ `imports` คือขอยืมคนจากแผนกอื่น `exports` คือยอมให้คนอื่นยืมของเรา สองตัวหลังต้องตรงกันเป็นคู่เสมอ

4 ข้อ 10 — DTO ป้องกันอะไร

ตอบสั้นๆ ก่อน

DTO คือ **ใบตรวจของด่านหน้า** ทีบอกว่าข้อมูลจากผู้ใช้ต้องหน้าตาแบบไหนถึงจะผ่านเข้ามาได้

ต้องมี เพราะ **TypeScript หายไปหมดตอนโปรแกรมทำงานจริง**

ทำไม TypeScript ช่วยไม่ได้

TypeScript ตรวจตอนทีเรากำลังเขียนโค้ดอยู่ พอแปลงเป็น JavaScript ทีรันจริงบนเซิร์ฟเวอร์
ทุกอย่างทีเกียวกับชนิดข้อมูลจะหายไปหมด

แต่ข้อมูลจากผู้ใช้มาถึงตอนโปรแกรมทำงานอยู่ และเป็นข้อความดิบๆ ทีใครส่งอะไรมาก็ได้

TypeScript จึงไม่มีทางกันได้ เพราะตอนนั้นมันไม่อยู่แล้ว

นีคือเหตุผลที DTO เขียนเป็น `class` ไม่ใช่ `interface` — เพราะ `class` ยังเหลืออยู่ใน JavaScript ส่วน `interface` หายไป

ตัวอย่างในโปรเจกต์

```
export class CreateAppointmentDto {
  @IsUUID()
  petId: string;

  @IsUUID()
  clinicId: string;

  @MinDate(() => new Date(), { message: 'appointmentDate must be in the future' })
  @Type(() => Date)
  @IsDate()
  appointmentDate: Date;

  @IsOptional()
  @IsString()
  @Trim()
  notes?: string;
}
```

DTO ทำ 3 อย่าง

หนึง ตรวจว่าค่าถูกต้องไหม

`@IsUUID()` ตรวจว่าเป็นรหัสรูปแบบทีถูกต้องจริง ไม่ใช่แค่เป็นตัวอักษรอะไรก็ได้

`@MinDate()` ตรวจว่าเป็นเวลาในอนาคต จองย้อนหลังไม่ได้

สอง แปลงชนิดข้อมูลให้

ข้อมูลทีส่งมาทางอินเทอร์เน็ตเป็นข้อความล้วน ไม่มีชนิด "วันเวลา"

`@Type(() => Date)` แปลงข้อความให้กลายเป็นวันเวลาจริงก่อนส่งต่อ

สาม ตัดของทีไม่ได้ขอทิง

ตั้งค่าไว้ที `main.ts` ครั้งเดียว ใช้ได้ทั้งระบบ

```
app.useGlobalPipes(
  new ValidationPipe({ transform: true, whitelist: true }),
);
```

`whitelist: true` แปลว่า field ไหนที่ไม่ได้ประกาศไว้ใน DTO จะถูกลบทิ้งก่อนถึงโค้ดของเรา

ถ้าไม่มี DTO จะเกิดอะไร

หน้าเว็บพังด้วย error 500 — ส่ง `petId` เป็นคำว่า `"abc"` มา ฐานข้อมูลจะโยน error ทันที ผู้ใช้เห็นหน้าจอพังแทนที่จะเห็นข้อความบอกว่ากรอกผิด

คำนวณเวลาผิดทั้งระบบ — วันเวลายังเป็นข้อความอยู่ พอเอาไปคำนวณช่องเวลาจะเพี้ยนหมด

จองย้อนหลังได้ — คนที่ยิง API เองจองวันที่ผ่านไปแล้วได้

ปัญหาที่หนักที่สุด คือส่งค่าที่ไม่ควรส่งเข้ามาได้ เช่น

```
{ "petId": "...", "clinicId": "...", "status": "CONFIRMED" }
```

`status` ไม่ควรมาจากผู้ใช้ ควรมาจากคลินิกเท่านั้น

ถ้าไม่มี `whitelist` ระบบจะบันทึกตามที่ส่งมา แปลว่าผู้ใช้ยืนยันนัดให้ตัวเองได้ **ข้ามการอนุมัติของคลินิกทั้งขั้นตอน** ซึ่งทำลายกระบวนการทำงานหลักของระบบทั้งหมด

> **สรุปข้อ 10**
> DTO คือด่านเดียวที่ตรวจข้อมูลผู้ใช้ตอนโปรแกรมทำงานจริง ทำสามอย่างคือตรวจค่า แปลงชนิด และตัดของที่ไม่ได้ขอทิ้ง TypeScript แทนไม่ได้เพราะมันหายไปตอนรัน และความเสียหายหนักสุดถ้าไม่มีคือผู้ใช้แนบ `status` มาเองแล้วยืนยันนัดให้ตัวเอง

5 ข้อ 11 — Dependency Injection คืออะไร ดียังไง

ตอบสั้นๆ ก่อน

แทนทีแต่ละคลาสจะสร้างของทีตัวเองต้องใช้ขึนมาเอง ให้ **บอกไปว่าต้องการอะไร แล้ว Nest จัดหาให้**

เหมือนบ้านไม่ต้องขุดบ่อน้ำเอง แค่ต่อท่อประปาทีมีอยู่แล้ว

หน้าตาในโค้ดจริง

```
@Injectable()
export class AppointmentService {
  constructor(private readonly prisma: PrismaService) {}
}
```

บรรทัดนีแปลว่า "ฉันต้องการ `PrismaService` นะ" แค่นัน ไม่ได้บอกว่าจะเอามาจากไหน

Nest เห็นแล้วจัดหาให้เอง

ถ้าสร้างเองจะเกิดอะไร

```
export class AppointmentService {
  private prisma = new PrismaService();
}
```

ปัญหาทีหนึง — ฐานข้อมูลจะรับไม่ไหว

ทุกครังทีสร้าง `PrismaService` ใหม่ มันจะเปิดช่องทางเชือมต่อไปหาฐานข้อมูลชุดใหม่ทังชุด

โปรเจกต์นีมี 5 คลาสทีใช้ฐานข้อมูล ก็จะได้ 5 ชุด ทังทีควรมีชุดเดียวใช้ร่วมกัน ฐานข้อมูลจะเต็มเร็วมาก

ปัญหาทีสอง — ต้องประกอบเองทุกที

`AuthService` ต้องการของ 3 อย่าง

```
constructor(
  private readonly userService: UserService,
  private readonly bcryptService: BcryptService,
  private readonly accessTokenService: AccessTokenService,
) {}
```

แต่ละอย่างก็ต้องการของตัวเองอีกทอดหนึง `UserService` ต้องการฐานข้อมูลกับตัวอัปโหลดรูป `AccessTokenService` ต้องการตัวเซ็น JWT กับตัวอ่านค่าตังค่า

ถ้าประกอบเอง ต้องไล่เขียนเป็นชันๆ ทุกจุดทีใช้

ปัญหาทีสาม — ทดสอบไม่ได้เลย อันนีสำคัญทีสุด

เทสต์ 54 ตัวของระบบจำกัดทีนังเขียนได้เพราะยัดฐานข้อมูลปลอมเข้าไปแทนของจริงได้

```
const service = new AppointmentService(fakePrisma);
```

ทำให้ทดสอบกฎ 2 คนต่อชัวโมงได้โดยไม่ต้องเปิดฐานข้อมูลจริง

ถ้าเขียน `new` ไว้ข้างในคลาส จะสลับไม่ได้เลย ต้องเปิด PostgreSQL จริงทุกครั้งทีอยากเทสต์

ปัญหาทีสี — เปลียนของยาก

วันหนึงถ้าย้ายจาก Cloudinary ไปใช้ทีอืน แบบนีแก้บรรทัดเดียว แต่ถ้า `new` เอง ต้องไล่แก้ทุกไฟล์

DI Container คือใคร

คือส่วนของ Nest ทีทำหน้าทีจัดหาของให้ ทำงาน 4 ขั้น

หนึง อ่านว่าแต่ละคลาสต้องการอะไรบ้าง จากทีเขียนไว้ใน constructor

สอง หาว่าใครลงทะเบียนของชินนันไว้ ดูในแผนกตัวเองก่อน ถ้าไม่มีค่อยดูแผนกทียืมมา

สาม สร้างตามลำดับ ของทีไม่ต้องพึงใครก่อน แล้วไล่ขึนมา

สี เก็บไว้ชินเดียวต่อทังแอป ใครขอก็ได้ชินเดิม `PrismaService` จึงมีแค่ตัวเดียวทังระบบ

ถ้าหาของไม่เจอ จะหยุดตังแต่ตอนเปิดแอป ไม่ปล่อยให้ไปพังตอนมีผู้ใช้จริง

เทียบให้เห็นชัด

	ใช้ DI	สร้างเอง (new)
จำนวนทีสร้าง	ชินเดียวใช้ร่วมกัน	ทุกทีสร้างใหม่
ทดสอบ	ยัดของปลอมได้	ต้องใช้ของจริง
เปลียนของ	แก้ทีเดียว	ไล่แก้ทุกไฟล์
รู้ว่าพัง	ตอนเปิดแอป	ตอนผู้ใช้เรียก

> **สรุปข้อ 11**
> แทนทีจะสร้างของใช้เอง ให้บอกว่าต้องการอะไรแล้วให้ Nest จัดหาให้
> ได้ผลสามอย่างคือของชินเดียวใช้ร่วมกันทังระบบไม่เปลืองการเชือมต่อฐานข้อมูล
> สลับของปลอมเข้าไปตอนเขียนเทสต์ได้ และถ้าประกอบไม่ครบจะรู้ตังแต่เปิดแอป

6 ข้อ 12 — ทำไม Controller ไม่ควรเขียน Business Logic

ตอบสั้นๆ ก่อน

Controller คือ **พนักงานต้อนรับ ไม่ใช่ช่าง**

หน้าที่คือรับเรื่องจากลูกค้า แปลงเป็นภาษาที่คนในบริษัทเข้าใจ แล้วส่งต่อให้คนที่ทำงานจริง

ส่วนงานจริงทั้งหมดอยู่ที่ Service

Controller ในโปรเจกต์นี้เป็นบรรทัดเดียวทุกตัว

```
@UseGuards(RolesGuard)
@Roles(Role.OWNER)
@Post('appointments')
create(
  @CurrentUser('sub') ownerId: string,
  @Body() dto: CreateAppointmentDto,
): Promise<AppointmentResponseDto> {
  return this.appointmentService.create(ownerId, dto);
}
```

ไม่มีเงื่อนไข ไม่มีการคำนวณ ไม่แตะฐานข้อมูลเลย

หน้าที่จริงมีแค่ 3 อย่าง

หนึ่ง แปลง request ให้เป็นตัวแปรธรรมดา — ดึงว่าใครเป็นคนขอจากบัตรผ่าน ดึงข้อมูลที่กรอกมาเป็น dto

สอง ประกาศว่าใครเข้าได้ ผ่าน `@Roles()`

สาม ประกาศว่าจะคืนข้อมูลหน้าตาแบบไหน

เท่านั้น

ถ้าใส่งานจริงลงไปจะเกิดอะไร

ทดสอบยากขึ้นมาก — ถ้ากฎ 2 คนต่อชั่วโมงอยู่ใน Controller การจะทดสอบต้องยิง request จริง ต้องมีบัตรผ่านจริง ต้องผ่านยามก่อน

แต่พอกฎอยู่ใน Service เรียกฟังก์ชันตรงๆ ได้เลย

เรียกได้ทางเดียว — โค้ดใน Controller ถูกเรียกได้ผ่าน request จากอินเทอร์เน็ตเท่านั้น ส่วน Service เรียกจากที่ไหนก็ได้

ในโปรเจกต์มีสถานการณ์แบบนี้จริง ตอนเจ้าของเก็บสัตว์เข้าคลัง ระบบต้องยกเลิกนัดที่ยังไม่ถึงของสัตว์ตัวนั้นด้วย ถ้าโค้ดยกเลิกนัดไปอยู่ใน Controller จะไม่มีทางเรียกจากตรงนั้นได้เลย

แก้ทีเดียวไม่จบ — สมมติเปลี่ยนจาก 2 คนเป็น 3 คนต่อชั่วโมง ตอนนี้แก้ค่าเดียว

```
export const MAX_APPOINTMENTS_PER_SLOT = 2;
```

ถ้ากฎกระจายอยู่หลายที ต้องไล่หาให้ครบ พลาดทีเดียวก็ทำงานไม่ตรงกัน

ครอบ transaction ไม่ได้ — การจองต้องนับที่ว่างและบันทึกลงฐานข้อมูลให้เป็นก้อนเดียวกัน
โค้ดแบบนี้ต้องอยู่ในชั้นที่รู้จักฐานข้อมูล ไม่ใช่ Controller

ผูกกับเว็บตลอดไป — วันหนึ่งถ้าอยากเพิ่มการนำเข้านัดจากไฟล์ หรือมีระบบเตือนล่วงหน้าอัตโนมัติ จะเรียก Service ได้ทันที แต่เรียก Controller ไม่ได้

การแบ่งชั้นในโปรเจกต์

ชั้น	รับผิดชอบ	ตัวอย่าง
Guard	ใครเข้าได้	ดูบัตรผ่าน ดูบทบาท
Pipe	ข้อมูลถูกรูปแบบไหม	ตรวจกับ DTO
Controller	แปลง request เป็นตัวแปร	ดึงว่าใครเป็นคนขอ
Service	งานจริงทั้งหมด	นับที่ว่าง ตรวจความเป็นเจ้าของ
Prisma	คุยกับฐานข้อมูล	นับ บันทึก

แต่ละชั้นเปลี่ยนได้โดยไม่กระทบชั้นอื่น วันหนึ่งถ้าเปลี่ยนวิธีรับ request ทั้งหมด ก็แก้แค่ Controller ส่วนกฎการทำงานยังใช้ได้เหมือนเดิม

> **สรุปข้อ 12**
> Controller เป็นพนักงานต้อนรับ หน้าที่คือแปลง request เป็นตัวแปรแล้วส่งต่อ ส่วนงานจริงอยู่ที่ Service เพราะ Service ทดสอบได้โดยไม่ต้องยิง request ครอบ transaction ได้ และเรียกจากที่อื่นได้ ผลที่เห็นชัดคือกฎ 2 คนต่อชั็วโมงอยู่ที่เดียว แก้ค่าเดียวจบ

7 ข้อ 13 — Request เข้ามาแล้วผ่านอะไรบ้าง

เทียบเป็นการเข้าตึกสำนักงาน

ขั้น	ในตึก	ใน NestJS
1	ยามหน้าประตูขอดูบัตร	Guard
2	ตรวจว่าเอกสารที่ถือมากรอกครบไหม	Pipe
3	พนักงานต้อนรับรับเรื่อง	Controller
4	ส่งให้แผนกที่ทำงานจริง	Service

ลำดับเต็ม

```
Request
  -> Guard
  -> Pipe
  -> Controller
  -> Service
  -> Response
```

ตัวอย่างจริง ตอนกดจองนัด

ขั้นที่ 1 ยามใหญ่ — `AuthGuard` ยืนอยู่หน้าทุกประตูทั้งตึก ขอดูบัตรผ่านก่อน

```
const [bearer, token] = request.headers.authorization?.split(' ') ?? [];
if (bearer !== 'Bearer' || !token) {
  throw new BadRequestException('Invalid authorization header');
}
const payload = await this.accessTokenService.verify(token);
request.user = payload;
```

ตรวจว่าลายเซ็นบนบัตรถูกต้องและยังไม่หมดอายุ แล้วจดไว้ว่าคนนี้เป็นใคร

ขั้นที่ 2 ยามหน้าห้อง — `RolesGuard` ดูว่าคนนี้มีสิทธิ์เข้าห้องนี้ไหม ห้องจองนัดเข้าได้เฉพาะเจ้าของสัตว์ ถ้าเป็นพนักงานคลินิกจะถูกปฏิเสธ

ขั้นที่ 3 ตรวจเอกสาร — `ValidationPipe` ตรวจข้อมูลที่กรอกมากับใบตรวจ DTO แปลงข้อความเป็นวันเวลา และตัดของที่ไม่ได้ขอทิ้ง

ขั้นที่ 4 พนักงานต้อนรับ — Controller ดึงว่าใครเป็นคนขอ จากที่ยามจดไว้ตอนขั้นที่ 1 แล้วส่งต่อให้ Service

ขั้นที่ 5 ทำงานจริง — Service ตรวจว่าสัตว์ตัวนี้เป็นของคนนี้จริงไหม ยังไม่ถูกเก็บเข้าคลังใช่ไหม แล้วเข้าไปนับที่ว่างในฐานข้อมูล

ทำไมยามต้องมาก่อนตรวจเอกสาร

เหตุผลด้านความปลอดภัย — ถ้าตรวจเอกสารก่อน คนที่ไม่มีบัตรจะได้รับข้อความว่า "ขาด field ชื่อ petId" กลับไป

ซึ่งเท่ากับบอกใบ้ให้คนนอกรู้ว่า API ของเราต้องการอะไรบ้าง เป็นการเปิดเผยโครงสร้างระบบฟรีๆ

เหตุผลด้านความเร็ว — ไม่ต้องเสียเวลาตรวจเอกสารกองใหญ่ของคนทียังไงก็เข้าไม่ได้อยู่แล้ว

จุดทีมักเข้าใจผิด

ยามตอบได้แค่ "เป็นใคร" กับ "บทบาทอะไร" เพราะข้อมูลสองอย่างนี้อยู่บนบัตรผ่าน

ส่วนคำถามทีต้องเปิดฐานข้อมูลดู อย่าง **"สัตว์ตัวนี้เป็นของคุณหรือเปล่า"** ยามตอบไม่ได้ ต้องไปตอบใน Service

```
if (pet.ownerId !== ownerId) {
  throw new ForbiddenException('This pet is not yours');
}
```

เรืองข้อความ error

โปรเจกต์นี้ไม่ได้เขียนตัวจัดการ error เอง ใช้ของมาตรฐานที Nest มีให้

นีคือเหตุผลทีพอ Service บอกว่า "ช่วงเวลานี้เต็มแล้ว" ข้อความนั้นเดินทางกลับไปถึงหน้าเว็บได้เอง
แล้วฟอร์มเอาไปแสดงให้ผู้ใช้เห็นแทนทีจะพัง

> **สรุปข้อ 13**
> เหมือนเข้าตึก — ยามดูบัตรก่อน (Guard) แล้วตรวจเอกสาร (Pipe) แล้วพนักงานต้อนรับรับเรือง (Controller)
> แล้วส่งให้แผนกทีทำงานจริง (Service) ต้องเรียงแบบนีเพราะไม่ควรเสียแรงตรวจเอกสารของคนทีเข้าไม่ได้
> และไม่ควรบอกใบ้โครงสร้าง API ให้คนทียังไม่ได้แสดงตัว

8 ข้อ 14 — unknown ปลอดภัยกว่า any ยังไง

ตอบสั้นๆ ก่อน

`any` แปลว่า **"ไม่ต้องตรวจ เชื่อฉัน"**

`unknown` แปลว่า **"ยังไม่รู้ว่าเป็นอะไร ต้องตรวจก่อนใช้"**

เทียบเป็นกล่องพัสดุ

มีกล่องมาส่งที่บ้าน ยังไม่ได้เปิดดู

`any` = เดินไปเตะกล่องเลยโดยไม่เปิดดู ถ้าข้างในเป็นแก้วก็แตก

`unknown` = ต้องเปิดดูก่อนว่าข้างในคืออะไร ถึงจะตัดสินใจได้ว่าจะทำอะไรกับมัน

เห็นความต่างในสองบรรทัด

```
let a: any = 5;
a.trim();

let u: unknown = 5;
u.trim();
```

บรรทัด	ตอนเขียนโค้ด	ตอนโปรแกรมทำงาน
`a.trim()`	ผ่านฉลุย	**พัง** เพราะตัวเลขไม่มี trim
`u.trim()`	**ฟ้องทันที**	ไปไม่ถึง เพราะแก้ตั้งแต่ตอนเขียนแล้ว

`trim()` เป็นคำสั่งตัดช่องว่างของข้อความ ตัวเลขไม่มีคำสั่งนี้

ตัวอย่างจริงในโปรเจกต์

```
export function TrimToNull() {
  return Transform(({ value }: { value: unknown }) => {
    if (typeof value !== 'string') return value;
    const trimmed = value.trim();
    return trimmed === '' ? null : trimmed;
  });
}
```

ฟังก์ชันนี้ทำหน้าที่ตัดช่องว่างหัวท้าย ถ้าเหลือว่างเปล่าให้กลายเป็นค่าว่าง

ทำไมต้องใช้ unknown ตรงนี้

เพราะฟังก์ชันนี้เอาไปใช้กับ field ไหนก็ได้ และค่าที่เข้ามาคือสิ่งที่ผู้ใช้ส่งมา

ผู้ใช้ส่งอะไรมาก็ได้ — ข้อความ ตัวเลข ค่าว่าง หรือแม้แต่ท้งก้อนข้อมูล เราไม่มีทางรู้ล่วงหน้า

`unknown` พูดความจริงข้อนี้ตรงๆ แล้ว TypeScript จะ **บังคับ** ให้เขียนบรรทัดตรวจก่อน ข้ามไม่ได้

```
if (typeof value !== 'string') return value;
```

หลังบรรทัดนี้ TypeScript จะรู้เองว่าทีเหลือเป็นข้อความแน่นอนแล้ว จึงเรียก `.trim()` ได้อย่างปลอดภัย

ถ้าเปลียนเป็น any จะเกิดอะไร

โค้ดจะคอมไพล์ผ่าน ไม่มีอะไรเตือนเลย

แต่พอมีคนกรอกเบอร์โทรเป็นตัวเลข `123456` แทนทีจะเป็นข้อความ จะพังทันทีด้วย

```
TypeError: value.trim is not a function
```

ผู้ใช้เห็นหน้าจอ error 500 ทั้งทีควรจะเห็นแค่ข้อความบอกว่ากรอกผิดรูปแบบ

อันตรายอีกอย่างของ `any` คือมันแพร่กระจาย ค่าทีได้จากตัวแปร `any` ก็กลายเป็น `any` ต่อไปเรือยๆ
พืนทีทีไม่มีการตรวจสอบเลยขยายวงกว้างขึนโดยไม่มีอะไรบอก

ทีอืนในโปรเจกต์ทีใช้ unknown

ค่าทีอ่านจากไฟล์ตังค่า ยังไม่ผ่านการตรวจ

```
export function validate(config: Record<string, unknown>) { ... }
```

ข้อมูลทีจะส่งไป API รูปร่างยังไม่แน่นอน

```
body?: Record<string, unknown> | FormData;
```

หลักทีใช้

ทุกอย่างทีมาจากนอกระบบ ให้ประกาศเป็น `unknown` **แล้วตรวจก่อนใช้**

`unknown` ไม่ได้ทำให้โค้ดปลอดภัยขึนเอง แต่มัน **บังคับให้เราเขียนการตรวจสอบ** ถ้าลืมก็คอมไพล์ไม่ผ่าน ซึงเป็นสิงที `any` ไม่มีให้

> **สรุปข้อ 14**
> `any` คือปิดการตรวจสอบทิง ส่วน `unknown` คือยังไม่รู้ว่าเป็นอะไร ต้องตรวจก่อนใช้ และคอมไพเลอร์จะไม่ยอมให้ข้าม จึงควรใช้ `unknown` กับทุกค่าทีมาจากผู้ใช้ อย่างใน `TrimToNull` ทีต้องเช็คว่าเป็นข้อความก่อน ถึงจะเรียก `.trim()` ได้

9 ข้อ 15 — Union Type กับ Enum ต่างกันยังไง

ตอบสั้นๆ ก่อน

ทั้งสองอย่างทำหน้าที่เดียวกันคือ **จำกัดว่าค่านี้เป็นอะไรได้บ้าง**

ต่างกันที่ **enum ยังอยู่ตอนโปรแกรมทำงาน แต่ union type หายไป**

เห็นความต่างชัดที่สุดในโปรเจกต์นี้

เพราะ PetWell ใช้ทั้งสองแบบ กับข้อมูลชุดเดียวกันเป๊ะ

ฝั่ง NestJS ใช้ enum

```
enum Role {
  OWNER
  CLINIC_STAFF
}
```

ฝั่ง Next.js ใช้ union type

```
export type Role = 'OWNER' | 'CLINIC_STAFF';
export type ClinicType = 'GENERAL' | 'EXOTIC' | 'EMERGENCY_24HR';
export type RecordType = 'VACCINE' | 'TREATMENT';
```

ค่าเหมือนกันทุกตัว แต่เลือกคนละแบบด้วยเหตุผลที่ต่างกัน

ตารางเปรียบเทียบ

	Union Type	Enum
ยังอยู่ตอนรัน	ไม่ หายไปเลย	อยู่ เป็นของจริง
เพิ่มขนาดไฟล์	0 ไบต์	มีโค้ดเพิ่มมา
เอาค่าทั้งหมดมาวนลูป	ไม่ได้	ได้
เขียนค่า	`'OWNER'` ตรงๆ	ต้อง `Role.OWNER`
เปลี่ยนชื่อทั้งโปรเจกต์	ต้องหาแทนที่เอง	เครื่องมือช่วยได้

ทำไม Backend เลือก Enum

หนึ่ง Prisma สร้างให้อัตโนมัติ และ PostgreSQL เก็บเป็นชนิดข้อมูลจริง ฐานข้อมูลจะปฏิเสธค่าแปลกปลอมตั้งแต่ชั้นล่างสุด

สอง ต้องใช้ค่าตอนโปรแกรมทำงานจริงๆ ตัวตรวจสอบข้อมูลต้องอ่านรายการค่าที่ถูกต้องออกมาดู

```
export class UpdateAppointmentStatusDto {
  @IsEnum(AppointmentStatus)
  status: AppointmentStatus;
}
```

`@IsEnum` ทำงานตอนมี request เข้ามาจริง ถ้าเป็น union type จะไม่มีอะไรให้มันอ่านเลย
เพราะหายไปตั้งแต่ตอนคอมไพล์แล้ว

สาม ใช้เป็นตารางกฎได้ ตารางนี้บอกว่าจากสถานะหนึ่ง ไปสถานะไหนได้บ้าง

```
const ALLOWED_TRANSITIONS: Record<AppointmentStatus, AppointmentStatus[]> = {
  PENDING: [AppointmentStatus.CONFIRMED, AppointmentStatus.REJECTED],
  CONFIRMED: [AppointmentStatus.COMPLETED, AppointmentStatus.CANCELLED],
  REJECTED: [],
};
```

ข้อดีคือถ้าวันหนึ่งเพิ่มสถานะใหม่แล้วลืมกำหนดกฎให้มัน TypeScript จะฟ้องทันที **บังคับให้เขียนครบทุกสถานะ**

ทำไม Frontend เลือก Union Type

หนึ่ง ข้อมูลที่ได้จาก API เป็นข้อความอยู่แล้ว union type อธิบายมันได้ตรงตัวโดยไม่ต้องแปลงอะไร

สอง ไม่เพิ่มขนาดไฟล์ที่ผู้ใช้ต้องดาวน์โหลด

สาม เขียนสั้นกว่าและอ่านง่ายกว่า

```
const dashboardFor = (r?: string) =>
  r === 'CLINIC_STAFF' ? '/staff/dashboard' : '/owner/dashboard';
```

ถ้าเป็น enum ต้องนำเข้ามาก่อนถึงจะเทียบได้ ซึ่งไม่คุ้มสำหรับการเทียบค่าเดียว

ข้อควรระวังของ enum

enum แบบตัวเลขยอมรับเลขที่ไม่มีในรายการได้ ซึ่งผิดความคาดหมาย

```
enum Status { Active, Inactive }
const s: Status = 99;
```

บรรทัดล่างคอมไพล์ผ่าน ทั้งที่ 99 ไม่มีอยู่ในรายการเลย

ถ้าจะใช้ enum ควรใช้แบบข้อความเสมอ ซึ่ง enum ที่ Prisma สร้างให้ก็เป็นแบบข้อความอยู่แล้ว

วิธีเลือก

ต้องใช้ค่าตอนโปรแกรมทำงานไหม เช่นเก็บลงฐานข้อมูล เอามาวนลูป หรือให้ตัวตรวจสอบอ่าน

ถ้าใช่ เลือก enum

ถ้าแค่บอก TypeScript ว่าตัวแปรนี้เป็นค่าอะไรได้บ้าง เลือก union type เพราะเบากว่าและเขียนง่ายกว่า

การที่โปรเจกต์นี้ใช้คนละแบบสองฝั่ง ไม่ใช่ความไม่สม่ำเสมอ แต่เป็นการเลือกเครื่องมือให้ตรงกับงานของแต่ละฝั่ง

> **สรุปข้อ 15**
> enum ยังอยู่ตอนโปรแกรมทำงาน จึงเหมาะกับค่าที่ต้องใช้จริงตอนรัน เช่นเก็บลงฐานข้อมูลหรือให้ตัวตรวจสอบอ่าน ส่วน union type หายไปตอนคอมไพล์ จึงเหมาะกับการบอกชนิดเฉยๆ และไม่เพิ่มขนาดไฟล์ PetWell เลยใช้ enum ที่ backend และ union ที่ frontend กับข้อมูลชุดเดียวกัน